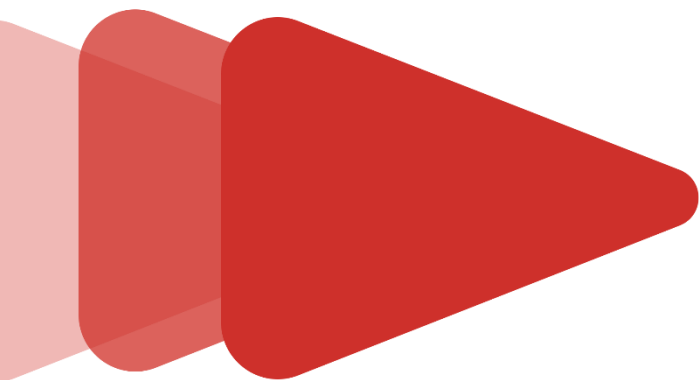




Arquitectura de la Solución Tecnológica Entrega y Recepción de Información Fiscal - Anexos

NAC-PTITARS26-00000017

***Gestión de Datos y Arquitectura Tecnológica
Departamento de Planificación TI y Gestión de Información -Datos
Dirección Nacional de Tecnología***

Junio 2026

Versión: 1.1.0

INFORMACIÓN DEL DOCUMENTO

Propósito:	El presente documento contiene las definiciones de arquitectura, estructura de componentes y su interacción para la Solución Tecnológica de Entrega y Recepción de Información Fiscal - Anexos.
Alcance:	Contempla las definiciones de arquitectura software, arquitectura de datos y arquitectura de infraestructura para el release 1 del proyecto que está desarrollando la aplicación de Anexos para la implementación integral del dominio funcional requerido para la administración, configuración, habilitación de anexos tributarios que permiten gestionar la estructura de anexos, la parametrización y catálogos operativos, la configuración de obligaciones, la habilitación de los anexos.
Audiencia:	Dirección Nacional de Tecnología

REVISIONES

Versión/Fecha	Autor	Descripción
1.0.0 / 19/12/2025	Santiago Simbaña	Emisión inicial para el release 1.
1.1.0 / 12/06/2026	Santiago Simbaña	Actualizaciones en el documento. - Actualización del nombre del proyecto. - Actualización de los diagramas de componentes externos, internos, despliegue. - Actualización del nombre del esquema de base de datos. - Actualización descripción de los componentes internos. - Componente de pistas de auditoria. - Versiones de dependencias y librerías.

Firmas de Responsabilidad

Fecha: 12/06/2026	Dirección Nacional de Tecnología	VERSIÓN 1.1.0	PÁGINAS 28
CARGO		FIRMA	
APROBADO POR:	Ing. Johanna Chagñay C. JEFE NACIONAL DE DEPARTAMENTO DE PLANIFICACIÓN TI Y GESTIÓN DE INFORMACIÓN-DATOS	 Validar únicamente en FirmaEC. Firmado electrónicamente por: JOHANNA ESPERANZA CHAGNAY CARPIO	
REVISADO POR:	Ing. Edison Cayambe Ch. COORDINADOR NACIONAL DE GESTIÓN DE DATOS Y ARQUITECTURA TECNOLÓGICA	 Validar únicamente en FirmaEC. Firmado electrónicamente por: EDISON ESTUARDO CAYAMBE CHICAIZA	
COLABORA:	Ing. Sofia Sevilla A. EXPERTO NACIONAL DE DESARROLLO	 Validar únicamente en FirmaEC. Firmado electrónicamente por: KAREN SOFIA SEVILLA AMAYA	
ELABORADO POR:	Ing. Santiago Simbaña H. EXPERTO NACIONAL DE ARQUITECTURA DE SOLUCIONES TECNOLÓGICAS	 Validar únicamente en FirmaEC. Firmado electrónicamente por: SANTIAGO DAVID SIMBAÑA HAYO	

Contenido

1.	Introducción	5
2.	Arquitectura de la Solución Tecnológica – Entrega y Recepción de Información Fiscal – Gestión Anexos.....	6
2.1.	Modelo de componentes externos y su interacción	6
2.1.1.	Solución Tecnológica	6
2.1.2.	Aplicación	6
2.2.	Arquitectura de software de la Solución Tecnológica – Entrega y Recepción de Información Fiscal – Gestión Anexos	8
2.2.1.	Definiciones de Arquitectura	8
2.2.2.	Modelo de componentes internos ambiente Intranet.....	11
2.3.	Arquitectura de datos de la Solución Tecnológica – Gestión Anexos	16
2.3.1.	Definiciones de Arquitectura	16
2.3.2.	Modelo de componentes internos	17
2.4.	Arquitectura de infraestructura de la Solución Tecnológica – Gestión Anexos	18
2.4.1.	Definiciones de Arquitectura	18
2.4.2.	Modelo de Despliegue Intranet – Gestión Anexos	19
3.	Glosario	21
3.1.	Acrónimos	21
3.2.	Definiciones	22
4.	Bibliografía	24
5.	Anexos	24
5.1.	Anexo A: Versión de tecnologías	24
5.2.	Anexo B: Matriz de integraciones y convivencias	26
5.3.	Anexo C: Estructura librería pistas-auditoria-lib v1.0.0.....	26

1. Introducción

El Sistema de Anexos Tributarios constituye un componente esencial dentro de la gestión de información fiscal del Servicio de Rentas Internas. La plataforma vigente, construida hace más de dos décadas, presenta limitaciones que afectan su evolución funcional, su capacidad para manejar la complejidad actual de los anexos y su alineación con los estándares tecnológicos institucionales. Las necesidades operativas recientes —incluyendo la definición dinámica de estructuras, la administración de catálogos, la configuración de obligaciones, la parametrización de reglas y la habilitación interna de anexos para contribuyentes— evidencian la importancia de modernizar el modelo de construcción y gobierno de estas definiciones.

El Release 1 (Intranet) se concentra exclusivamente en la construcción y habilitación de la plantilla de anexos, estableciendo las bases para los procesos posteriores de recepción, validación y procesamiento que se implementarán en releases siguientes. En este alcance, el sistema debe permitir:

- Definir y versionar la estructura del anexo (secciones, casilleros y metadatos).
- Administrar catálogos funcionales propios del dominio.
- Configurar la relación entre anexos y obligaciones tributarias.
- Parametrizar condiciones asociadas a la visualización de campos en la plantilla.
- Generar las integraciones con sistemas institucionales conforme las funcionalidades requeridas para: notificaciones, catastro, obligaciones y catálogos transversales.

La presente arquitectura se diseña bajo principios institucionales de resiliencia, desacoplamiento y estandarización, empleando microservicios alineados a DDD, exposición controlada mediante API Management, autenticación centralizada con Keycloak. Asimismo, se adopta un modelo de despliegue en una plataforma de contenedores on-premise, cumpliendo con los lineamientos de observabilidad, gobernanza y alta disponibilidad requeridos por el SRI.

Este documento consolida los componentes funcionales, las interacciones entre microservicios, los modelos de datos y las decisiones arquitectónicas que sustentan la implementación del Release 1 del Sistema de Anexos. El resultado es una plataforma modular, robusta y preparada para escalar en releases posteriores hacia la recepción masiva, validación operativa, procesamiento tributario y exposición para contribuyentes externos.

A continuación, se detalla la lista de la documentación de la solución tecnológica con sus respectivos repositorios:

DESCRIPCIÓN	ruta
Documentación de la arquitectura de la solución tecnológica.	Repositorio: ALFRESCO Repositorio> Direcciones Nacionales> Direccion Nacional de Tecnología> Departamento de Planificación TI> 01 ARQUITECTURA TECNOLÓGICA> 2. ARQUITECTURA TECNOLÓGICA DE APLICACIÓN, DATOS, INFRAESTRUCTURA> ARQUITECTURA DE SOLUCIÓN TECNOLÓGICA> 02 CADENA DE VALOR> 02 CONTROL AL CUMPLIMIENTO TRIBUTARIO> ENTREGA Y RECEPCIÓN DE INFORMACIÓN FISCAL> ANEXO
Diagramas/modelos de la arquitectura de la solución tecnológica Anexos.	Repositorio: TEAMWORK SERVER Categoría: Arquitectura de la Solución Ubicación: Data/CADENA DE VALOR/Control al Cumplimiento Tributario/Entrega y Recepción de Información Fiscal/ Gestión Anexos
Requerimiento de Usuario de la aplicación Anexos.	Repositorio: ALFRESCO Repositorio Direcciones Nacionales> Direccion Nacional de Tecnología> Departamento de Planificación TI> 01 ARQUITECTURAS, ESTÁNDARES Y GUÍAS> ARQUITECTURA TECNOLÓGICA DE LA SOLUCIÓN> 02 CADENA DE VALOR> 02 CONTROL AL CUMPLIMIENTO TRIBUTARIO> ENTREGA Y RECEPCIÓN DE INFORMACIÓN FISCAL> ANEXOS> CONSTRUCCION> EDITABLES> ARS_25027_Anexos_vE1.0.0

2. Arquitectura de la Solución Tecnológica – Entrega y Recepción de Información Fiscal – Gestión Anexos

2.1. Modelo de componentes externos y su interacción

2.1.1. Solución Tecnológica

Enlace del modelo UML y descripción de los Componentes Externos de la Solución tecnológica Entrega y Recepción de información Fiscal – Anexos:

Url confluence:

<http://sriargic01:1100/pages/viewpage.action?pageId=121208936> .

2.1.2. Aplicación

La aplicación tecnológica de Anexos se ha concebido como una aplicación que está disponible para su utilización desde los ambientes internet e intranet, cuyas principales interacciones pueden darse considerando las integraciones que se

muestran de manera gráfica de los componentes externos de la aplicación Entrega y Recepción de Información Fiscal.

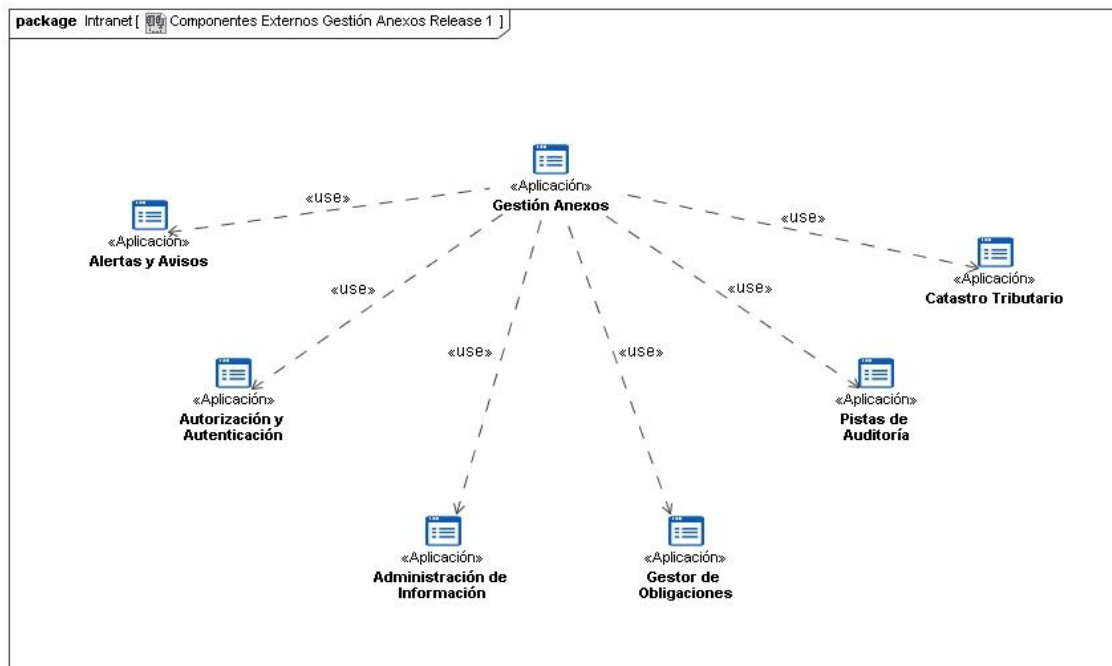


Figura 1: Modelo de Componentes Externos Entrega y Recepción de Información Fiscal - Anexos

Aplicación	Relación
Gestión Anexos	La aplicación Anexos constituye el núcleo funcional encargado de la gestión integral del ciclo de vida de los anexos tributarios en Internet e Intranet. Coordina la interacción con componentes transversales institucionales para obtener catálogos, contribuyentes, registrar el cumplimiento de las obligaciones, relacionada al tipo operativo del anexo, el registro de la pista de auditoría respectiva y emitir notificaciones. Las integraciones se realizan mediante servicios REST gestionados por el API Management, autenticados por el RedHat SSO(Keycloak) y operados por los microservicios `sri-anexos-*service`. La Aplicación Intranet en el primer Release permite la gestión interna de las plantillas de anexos, su configuración y ciclo de aprobación aprovisionando los servicios a ser utilizados en releases posteriores
Administración de Información	Proporciona los catálogos institucionales utilizados por la solución Anexos para el uso de parámetros, tipos de impuestos, tasas, sanciones y metadatos transversales. La interacción se realiza por servicios REST que actúan como fuente única de datos maestros, garantizando consistencia y gobernanza institucional.
Autenticación y Autorización	Servicio centralizado basado en Red Hat SSO/Keycloak, utilizado para la validación de accesos y permisos mediante tokens OAuth2/OpenID. El sistema Anexos valida cada solicitud, obtiene roles autorizados y garantiza la seguridad en los flujos tanto internos como externos.
Alertas y Avisos	Permite emitir notificaciones automáticas hacia funcionarios a través de correo institucional. El sistema gestión Anexos registra

	mensajes estructurados para informar el estado del ciclo de aprobación de la plantilla (solicitud de revisión, aceptado, devuelto con observaciones). La integración asegura trazabilidad y comunicación oportuna.
Catastro Contributivo	Repositorio oficial de datos del sujeto pasivo. El sistema Gestión Anexos consulta información del contribuyente (estado del RUC, razón social, tipo de contribuyente) para validar que el anexo sea presentado por el sujeto catastrado en la administración tributaria.
Gestor de Obligaciones	Provisiona el mecanismo para el registro y del cumplimiento de las obligaciones tributarias. El sistema Gestión Anexos comunicará en su momento, realizado el procesamiento exitoso de un anexo, la necesidad de que la obligación relacionada al Anexo como cumplida.
Pistas de Auditoria	El mecanismo transversal institucional para el registro de eventos auditables. El sistema Gestión Anexos registra eventos como autenticación, y cambios de estado, en función del requerimiento funcional con el objetivo de dar cumplimiento a normativas de seguridad y trazabilidad.

2.2. Arquitectura de software de la Solución Tecnológica – Entrega y Recepción de Información Fiscal – Gestión Anexos

2.2.1. Definiciones de Arquitectura

Definición: Utilizar un JDK compatible con OpenJDK, conforme a la versión establecida en el estándar institucional vigente. Para el presente release, se adopta GraalVM Community Edition como distribución del JDK para la ejecución de los servicios, garantizando el cumplimiento del estándar Java definido

Motivación: Se asegura la compatibilidad, estabilidad y soporte institucional en los entornos de ejecución, así como el cumplimiento de los lineamientos corporativos de seguridad, mantenimiento y arquitectura resiliente, establecidos en el Estándar de Backend – Arquitectura resiliente. Adicionalmente, la adopción de GraalVM permite optimizar el tiempo de arranque y el consumo de recursos en entornos contenerizados, manteniendo la portabilidad, gobernanza tecnológica y alineación con el estándar Java institucional.

Definición: Implementar Quarkus como framework backend para todos los servicios sri-anexos-*--service y adapters institucionales.

Motivación: Quarkus ofrece tiempos de arranque optimizados, menor huella de memoria y soporte nativo para entornos contenerizados, lo que mejora el rendimiento y escalabilidad de la arquitectura de microservicios.

Definición: Utilizar Vue.js como framework de frontend para el portal Intranet del Sistema Gestión Anexos.

Motivación: Proporciona un modelo reactivo, alta productividad en el desarrollo y una experiencia de usuario moderna, alineada con los lineamientos institucionales de interfaces web.

Definición: Implementar servicios REST para la exposición e integración de funcionalidades del dominio Anexos.

Motivación: Facilita la interoperabilidad entre equipos y sistemas heterogéneos mediante HTTP estándar, simplificando consumo, prueba, documentación y mantenimiento.

Definición: Definir y documentar todas las APIs RESTful con OpenAPI antes de su implementación.

Motivación: Establece contratos claros desde etapas tempranas, permite generación automática de artefactos (mocking, SDKs, pruebas) y asegura gobernanza de APIs bajo el api management.

Definición: Utilizar RH-SSO/Keycloak como proveedor único de identidad para autenticación en los módulos de Intranet e Internet.

Motivación: Centraliza la gestión de sesiones, roles y permisos, reduce vulnerabilidades y garantiza cumplimiento de estándares institucionales de seguridad.

Definición: Contenerizar todos los componentes (frontend, backend, adapters, workers/event listeners) y desplegarlos en la Plataforma de Contenedores.

Motivación: Aporta portabilidad, consistencia entre ambientes, autoscaling, monitoreo nativo y cumplimiento de la estrategia institucional de orquestación.

Definición: Todos los microservicios transaccionales del dominio Anexos deberán utilizar Oracle como motor de base de datos autorizado.

Motivación: Garantiza integridad, soporte institucional, alta disponibilidad y alineación con la arquitectura de datos institucional.

Definición: Se define el uso de Oracle Database 19c como base de datos relacional obligatoria para los componentes transaccionales de la solución Anexos.

Para estructuras JSON, considerando que Oracle 19c no dispone del tipo de dato nativo JSON, su almacenamiento deberá realizarse mediante tipos soportados como CLOB, BLOB o VARCHAR2, aplicando validación IS JSON, funciones SQL/JSON e indexación cuando corresponda.

Motivación: Permite mantener la alineación con la plataforma institucional disponible, aprovechando las capacidades relacionales de Oracle y su soporte para

manejo de documentos JSON en versión 19c, sin introducir motores adicionales ni asumir funcionalidades disponibles únicamente en versiones superiores, como el tipo nativo JSON de Oracle 19c o posterior. Esta decisión refuerza la mantenibilidad, eficiencia y alineación con estándares modernos de almacenamiento híbrido (relacional + semiestructurado).

Definición: Toda comunicación e integración con sistemas legados institucionales se realizará mediante servicios web REST de integración definidos para tal fin, los cuales serán consumidos por los microservicios. Los microservicios no deberán acceder directamente a bases de datos, artefactos, componentes internos o mecanismos propietarios de los sistemas legados, salvo excepción técnica formalmente aprobada.

Motivación: Esta decisión desacopla la solución Gestión Anexos de los sistemas legados, estandariza la integración, facilita monitoreo y auditoría, y reduce riesgos por dependencia tecnológica directa.

Definición: Todas las APIs expuestas deberán ser gestionadas y publicadas a través del API Management, el cual actuará como punto de entrada controlado para los consumidores.

Motivación: Permite una exposición segura, gobernada, monitoreable y alineada con las políticas institucionales, manteniendo además una arquitectura desacoplada entre la capa de gestión de APIs y la plataforma de ejecución de microservicios.

Definición: La solución Anexos aplicará los principios de Domain-Driven Design —DDD— para delimitar sus capacidades funcionales mediante bounded contexts, los cuales servirán como base para la definición de microservicios, modelos de dominio, entidades, objetos de valor, agregados y servicios de dominio, según corresponda a la complejidad funcional de cada componente.

Los eventos de dominio o integración deberán definirse únicamente cuando exista una necesidad real de comunicación desacoplada, trazabilidad funcional o propagación de cambios entre contextos

Motivación: Permite establecer límites funcionales claros, reducir acoplamiento entre componentes, mantener coherencia en el modelo de dominio y facilitar la evolución controlada. Asimismo, evita que los microservicios se definan únicamente por criterios técnicos o de operación, asegurando que su diseño responda a capacidades de negocio, responsabilidades bien delimitadas y reglas funcionales propias del dominio.

Definición: Todos los microservicios deberán integrar logging centralizado (EFK/ELK), métricas y trazabilidad distribuida.

Motivación: Permite diagnóstico, auditoría, supervisión activa y cumplimiento de normativas internas.

2.2.2. Modelo de componentes internos ambiente Intranet

El modelo de componentes internos del ambiente Intranet describe los elementos técnicos que conforman la solución del Sistema Gestión Anexos – Release 1, sus responsabilidades, dependencias (directas y transitivas) y las tecnologías utilizadas para brindar los servicios de cada componente.

Este modelo se basa en una arquitectura de microservicios orientada a dominio (DDD), apoyada en eventos (EDA), expuesta mediante API Management corporativo y despliegue de los componentes contenerizados sobre Kubernetes sobre infraestructura on-premise.

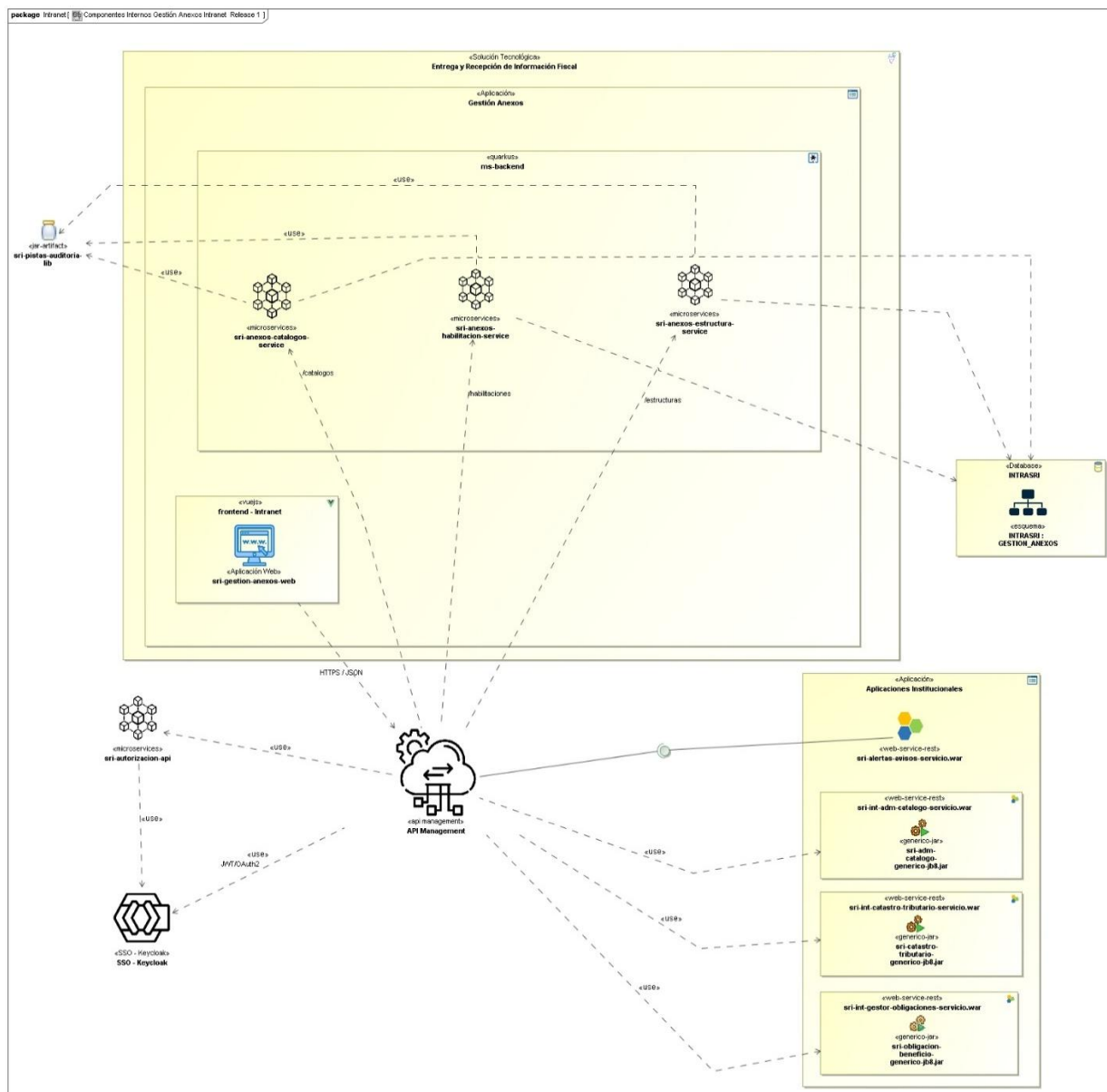


Figura 2: Modelo de Componentes Internos Intranet Entrega y Recepción de Información Fiscal – Gestión Anexos

Artefacto/Microservicio	Descripción	Tecnología Utilizada
Anexos:		
sri-anexos-estructura-service	Descripción: Gestiona la estructura de las plantillas de Gestión Anexos, desde su creación y configuración inicial hasta el control de sus estados dentro del flujo de construcción y solicitud de revisión. Tipo: Microservicio de dominio Bounded Context: Estructura de Anexos(definición de anexos, secciones, casilleros, versiones y estados de diseño) Agregados principales: Anexo, Sección, Casillero, VersionEstructura. Endpoints base: estructuras/api/v1/ Persistencia: Oracle INTRASRI, esquema GESTION_ANEXOS. Dependencias directas: API Management, Oracle INTRASRI	• Java JDK versión: Anexo A. • Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A • Hibernate Reactive • Panache Reactive • OIDC/Keycloak • MapStruct: Anexo A • OpenAPI • Bean Validation • JUnit + RestAssured • JaCoCo • Sonar/PMD
sri-anexos-catalogos-service	Descripción: Gestiona la creación y mantenimiento de los catálogos requeridos por la solución, incluyendo catálogos específicos, campos de información y mensajes. Estos elementos constituyen insumos base para la definición y construcción de las plantillas de Anexos dentro del proceso de Gestión de Anexos. Tipo: Microservicio de dominio Bounded Context: Catálogos de Anexos. Agregados principales: CatalogoAnexo, ValorCatalogoAnexo. Endpoints base: catalogos/api/v1/. Persistencia: Oracle INTRASRI, esquema GESTION_ANEXOS Dependencias directas: API Management, Oracle INTRASRI. Dependencias transitivas: Servicios de Anexos que leen estos catálogos (estructuras, parametrizaciones, habilitación).	• Java JDK versión: Anexo A. • Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A • Hibernate Reactive • Panache Reactive • OIDC/Keycloak • MapStruct: Anexo A • OpenAPI • Bean Validation • JUnit + RestAssured • JaCoCo • Sonar/PMD
sri-anexos-habilitacion-service	Descripción: Gestiona el proceso de revisión, aprobación y publicación de las plantillas de Gestión Anexos, incluyendo el manejo de los estados Devolver y Aprobar, así como el registro de las observaciones realizadas por el usuario responsable. Tipo: Microservicio de dominio Bounded Context: Habilitación de Anexos. Agregados principales: HabilitacionAnexo, EstadoHabilitacion, ObservacionHabilitacion. Endpoints base: habilitaciones/api/v1/. Persistencia: Oracle INTRASRI, esquema GESTION_ANEXOS. Dependencias directas: API Management, Oracle INTRASRI, sri-anexos-estructura-service, sri-anexos-catalogo-service.	• Java JDK versión: Anexo A. • Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A • Hibernate Reactive • Panache Reactive • OIDC/Keycloak • MapStruct: Anexo A • OpenAPI • Bean Validation • JUnit + RestAssured • JaCoCo • Sonar/PMD

FrontEnd Intranet		
Aplicativo Web sri-gestion-anexos-web	Aplicación web usada por usuarios internos para diseñar anexos, configurar parametrizaciones, obligaciones, gestionar. Consume APIs REST del dominio Anexos.	• Vue versión: Anexo A. • Node versión: Anexo A. • NPM versión: Anexo A. • Vue Cli versión: Anexo A. • TypeScript versión: Anexo A. • Vite versión: Anexo A. • PrimeVue versión: Anexo A. • Primeicons versión: Anexo A. • BootStrap versión: Anexo A. • Pinia versión: Anexo A. • Axios • Keycloak-js versión: Anexo A. • Formulajs: Anexo A. • vuelidate/core: Anexo A. • vuelidate/validators: Anexo A. • jexl: Anexo A. • uuid: Anexo A. • vuedraggable: Anexo A. • read-excel-file: Anexo A.
API Management		
API Management	Puerta de entrada única a todas las APIs del dominio Anexos e integraciones; aplica políticas de seguridad, rate limiting, trazabilidad y versionado. Aísla al dominio de consumidores y de detalles de exposición.	
Autenticación / SSO		
SSO / Keycloak	Proveedor de identidad corporativo; emite tokens OAuth2/OIDC usados por API Management y microservicios para validar identidad y roles.	SSO version: Anexo A.
sri-autorizacion-api	Descripción: Gestiona permisos y roles específicos del dominio Anexos, complementando la autenticación global de Keycloak. Tipo: Microservicio de dominio (seguridad de negocio)	• Java JDK versión: Anexo A. • Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A.

	Bounded Context: Seguridad del dominio Anexos (autorización funcional). Agregados principales: Rol, Permisos, AsignacionPermisosUsuario. Endpoints base: autorizacion/api/v1/. Persistencia: Oracle INTRASRI. Dependencias directas: API Management, Oracle INTRASRI, Keycloak. Dependencias transitivas: Resto de microservicios sri-anexos-* consultan o usan su librería para autorización fine-grained.	• Hibernate Reactive • Panache Reactive • OIDC/Keycloak • MapStruct: Anexo A • OpenAPI • Bean Validation • JUnit + RestAssured • JaCoCo • Sonar/PMD
Base de datos		
INTRASRI	La aplicación de anexos utiliza la base de datos INTRASRI para guardar/modificar/consultar toda la información transaccional de esta aplicación en intranet, para lo cual utiliza el esquema denominado GESTION_ANEXOS .	• Oracle Database version: Anexo A.
Aplicaciones institucionales		
sri-int-gestor-obligaciones-servicio	Descripción: Expone servicios del Gestor de Obligaciones: consulta de obligaciones tributarias (periodicidad, vencimiento, estado) y registro de obligación cumplida cuando se presenta el anexo. Tipo: Servicio Web REST de integración / fachada Gestor de Obligaciones.	• Java JDK versión: Anexo A. • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jakarta Restful Web Services: Anexo A
sri-int-adm-catalogo-servicio	Descripción: Provee acceso a las tablas transversales de Administración de Información (ADM Año, Mes, Países, Provincias, Cantones, Parroquias, Obligaciones, Identificaciones, etc.), con navegación jerárquica según el nivel requerido. Tipo: Servicio Web REST de integración / catálogos transversales.	• Java JDK versión: Anexo A. • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jakarta Restful Web Services: Anexo A
sri-int-catastro-tributario-servicio	Descripción: Expone servicios de consulta de información del contribuyente (RUC, razón social, medios de contacto, atributos adicionales) a partir del sistema legado de Catastro Tributario. Tipo: Servicio Web REST de integración / fachada Catastro Tributario.	• Java JDK versión: Anexo A. • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A

		• Jakarta Restful Web Services: Anexo A
sri-alertas-aviso-servicio	Descripción: Servicio Web que provee operaciones para enviar notificaciones internas (correos, avisos) usando la solución institucional de Alertas y Avisos: publicación de anexo, solicitud de revisión, rechazos, etc. Tipo: Servicio Web REST de integración / mensajería de Alertas y Avisos.	• Java JDK versión: Anexo A. • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jakarta Restful Web Services: Anexo A
Genéricos		
sri-adm-catalogo-generico-jb8.jar	Componente requerido por la aplicación para la obtención de datos de catálogos transversales que corresponden a la administración del SRI, tales como: usuarios, estructura organizacional.	• JDK versión: Anexo A • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jackson Anexo A
sri-catastro-tributario-generico-jb8.jar	Componente requerido por la aplicación para obtener los datos del registro único de contribuyentes esto es: número ruc, razón social, tipo y clase de contribuyente, jurisdicción, representante legal, fecha de inicio y reinicio de actividades, fecha inicio y fin de obligado a llevar contabilidad, fallecido y si es fantasma o no.	• JDK versión: Anexo A • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jackson Anexo A
sri-obligacion-beneficio-generico-jb8.jar	Componente requerido por la aplicación para obtener información de consulta de obligaciones tributarias y registro de obligación.	• JDK versión: Anexo A • Jakarta Enterprise Beans: Anexo A • Jakarta Contexts and Dependency Injection: Anexo A • Jakarta Persistence: Anexo A • Jackson Anexo A
Librería		
sri-pistas-auditoria-lib	La librería JAR moderniza el mecanismo de captura y transporte de pistas de auditoría para microservicios Quarkus sobre Kubernetes (VKS), preservando compatibilidad de campos con el ecosistema ELK existente y alineándose con el	• Java JDK versión: Anexo A. • Quarkus (REST JSON-B, Mutiny): Anexo A. • Lombok: Anexo A

	estándar institucional, reemplaza el artefacto legado sri-pista-auditoria-jb8.jar v2.5.0.	• Hibernate • Panache • OIDC/Keycloak • MapStruct: Anexo A • FluentBit (Plugin envío data a ELK) • Jakarta EE10 • OpenTelemetry • Jandex Maven Plugin • Definición Anexo C
--	---	--

2.3. Arquitectura de datos de la Solución Tecnológica – Gestión Anexos

2.3.1. Definiciones de Arquitectura

Definición: Utilizar Oracle 19c o superior como motor de base de datos relacional para el almacenamiento, gestión y persistencia de los datos transaccionales generados por la solución Gestión Anexos.

Motivación: Oracle Database constituye el motor relacional institucional utilizado para la gestión de información transaccional, ofreciendo capacidades de consistencia, integridad referencial, seguridad, disponibilidad, respaldo y recuperación.

Definición: La base de datos a ser utilizada por la aplicación de Gestión Anexos en Intranet es INTRASRI.

Motivación: Utilizar la base de datos establecida para las aplicaciones de intranet con la arquitectura vigente.

Definición: Repositorio transaccional primario en el entorno Intranet del SRI. Almacena toda la información operativa del ciclo de vida de los anexos: creación, edición, validación, auditoría, estados y control de usuarios. Toda escritura ocurre exclusivamente en esta base.

Motivación: Garantiza la integridad referencial, seguridad, transaccionalidad ACID y trazabilidad de datos críticos, en un entorno controlado y no expuesto directamente a internet. Alineado con el principio de “fuente de verdad” operativa.

Definición: Base de datos de solo lectura en el entorno Internet, sincronizada desde INTRASRI. Se utiliza para exponer datos procesados de los anexos a contribuyentes a través del portal SRI en Línea. Prohibido realizar escrituras directas.

Motivación: Separa ambientes operativos y públicos, evitando comprometer el entorno transaccional. Aumenta la seguridad, mejora el rendimiento de consulta y reduce la exposición directa de datos internos.

Definición: El esquema a ser utilizado por la aplicación es GESTION_ANEXOS.

Motivación: Dar cumplimiento a los estándares institucionales en cuanto al nombrado de objetos de bases de datos.

Definición: El diseño, construcción y mantenimiento de los objetos de base de datos de la solución Gestión Anexos Intranet deberá realizarse conforme a los estándares institucionales de base de datos, lineamientos de modelado, nomenclatura, seguridad, integridad, auditoría, administración de objetos, versionamiento de scripts y buenas prácticas definidas por la institución.

Motivación: Esta definición asegura que la base de datos de la solución mantenga consistencia técnica, trazabilidad, calidad, seguridad y facilidad de administración dentro del ecosistema institucional, evitando estructuras aisladas o criterios particulares que dificulten la operación, mantenimiento y evolución de la aplicación.

2.3.2. Modelo de componentes internos

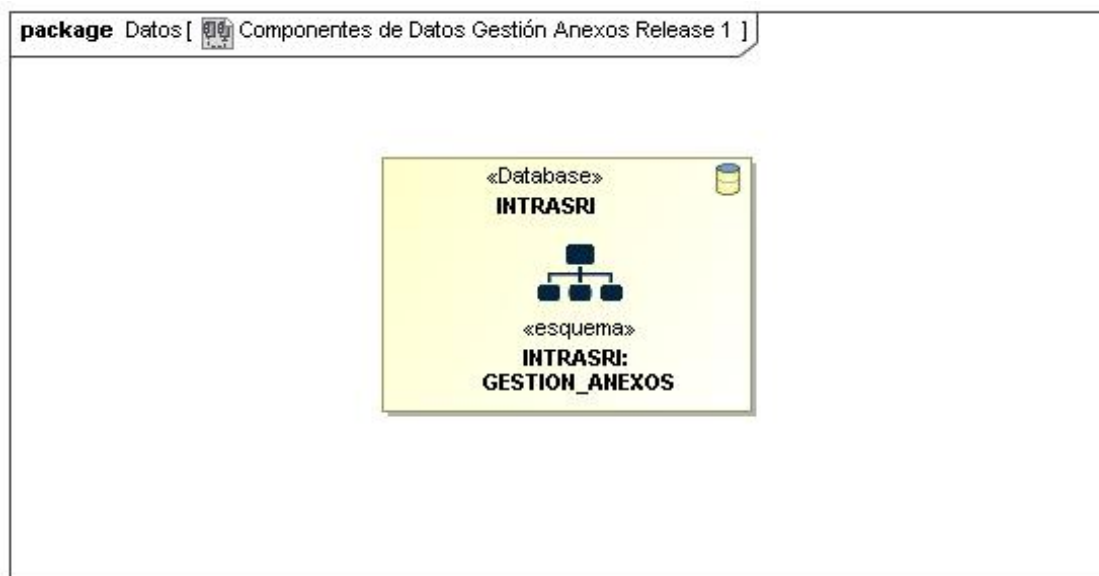


Figura 4: Modelo de Componentes Internos de Datos - nivel de diseño Intranet
Entrega y Recepción de Información Fiscal - Anexos

Diagrama de datos para las aplicaciones: Aplicación y su interacción con la(s) base de datos y esquema(s)

Componente	Descripción	Tecnología Utilizada
INTRASRI		
INTRASRI: GESTION_ANEXOS	Esquema de la aplicación Anexos donde se gestiona toda la información transaccional de esta aplicación en ambiente intranet.	Esquema de base de datos Oracle versión Anexo A .

2.4. Arquitectura de infraestructura de la Solución Tecnológica – Gestión Anexos

2.4.1. Definiciones de Arquitectura

Definición: La aplicación debe ser desplegada utilizando una plataforma de orquestación de contenedores, permitiendo una gestión eficiente, escalabilidad automática y alta disponibilidad del servicio.

Motivación: El uso de una plataforma de contenedores permite mejorar la resiliencia y disponibilidad de la aplicación, facilitando el despliegue continuo y la recuperación ante fallos. Dado que se trata de un servicio crítico, es fundamental garantizar su operatividad constante y su capacidad de escalar según la demanda de los contribuyentes.

Definición: La estrategia de alta disponibilidad y/o escalamiento horizontal a implementar para los componentes de la presente solución será a través del balanceo de instancias que trabajan en modo standalone, dicho balanceo será configurado en el componente F5 y en los servidores apache donde se configura el balanceo.

Motivación: Establecer el mecanismo que debe considerar la solución para el crecimiento horizontal y alta disponibilidad, cuando se lo requiera.

Definición: La solución Gestión Anexos Intranet será desplegada como cargas contenerizadas sobre una plataforma de orquestación Kubernetes, en infraestructura institucional on-premise. La alta disponibilidad y el escalamiento horizontal de los componentes se gestionarán mediante capacidades nativas de Kubernetes, tales como Deployments, ReplicaSets, Services, réplicas de pods, health checks, readiness/liveness probes y políticas de escalamiento, según corresponda.

Motivación: El uso de Kubernetes permite distribuir réplicas de los componentes, recuperar automáticamente pods ante fallos, balancear tráfico dentro del clúster y escalar horizontalmente según la demanda. Asimismo, la integración con API Management e Istio permite mantener una exposición controlada, segura, trazable y alineada con la arquitectura institucional definida.

Definición: Los componentes legados o institucionales que permanezcan fuera de Kubernetes mantendrán sus mecanismos actuales de despliegue, balanceo y operación, mientras que los componentes nuevos de Anexos deberán registrarse por el modelo contenerizado definido.

Motivación: Permite una convivencia controlada entre la arquitectura vigente y la arquitectura objetivo, evitando forzar migraciones no contempladas dentro del alcance del Release 1.

Definición: Los componentes de la solución deberán organizarse en namespaces de Kubernetes de acuerdo con criterios de dominio, ambiente, seguridad y operación definidos por la institución.

Motivación: Permite aislar recursos, aplicar políticas diferenciadas, facilitar administración operativa y mejorar el control sobre permisos, cuotas y despliegues.

Definición: Los componentes críticos de la solución deberán ejecutarse con múltiples réplicas de pods, distribuidas en nodos worker cuando la plataforma lo permita, evitando puntos únicos de falla a nivel de aplicación.

Motivación: Garantiza continuidad del servicio ante fallos de pod o nodo, permite mantenimiento controlado y mejora la capacidad de atención concurrente.

2.4.2. Modelo de Despliegue Intranet – Gestión Anexos

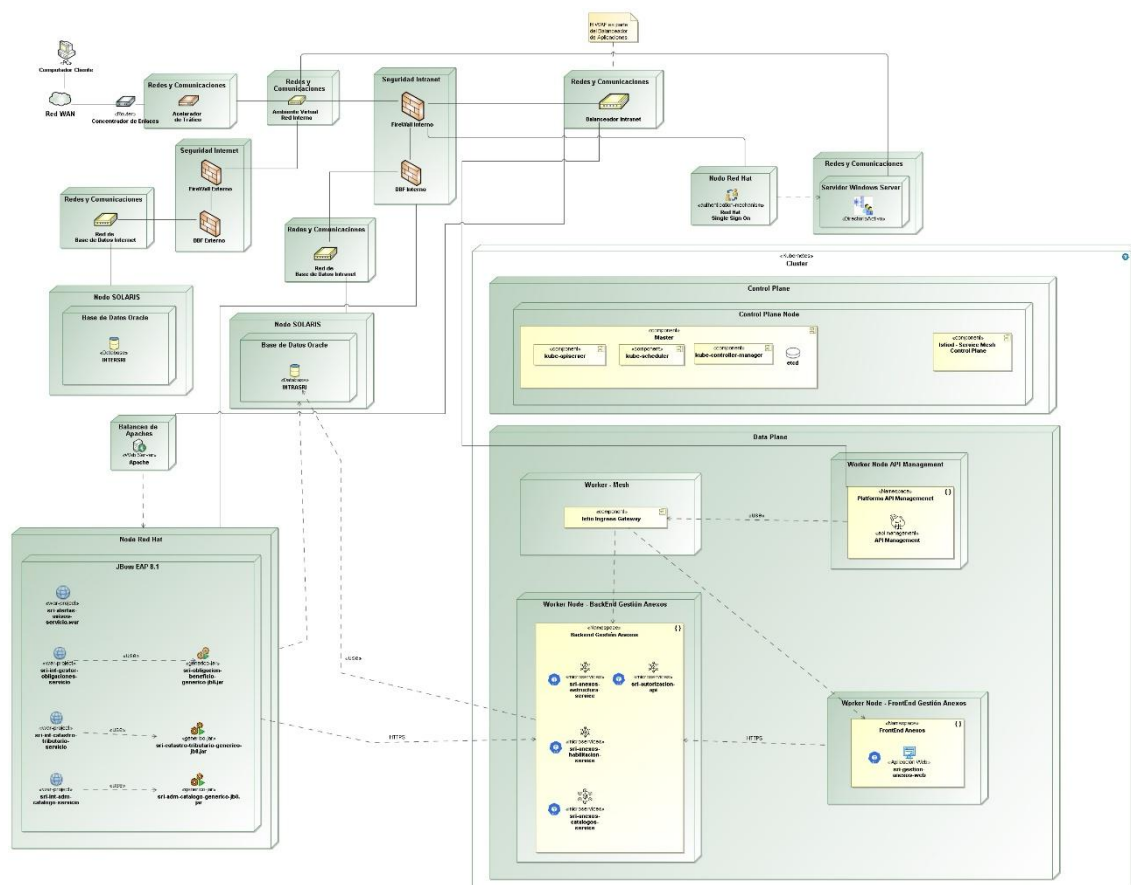


Figura 6: Modelo de Despliegue de la Solución Tecnológica Entrega y Recepción de Información Fiscal – Anexos

Nota:

- Los componentes de Seguridad Internet y Seguridad Intranet se encuentran descritos en la arquitectura general “Arquitectura Tecnológica Seguridad Informática”.

- Los componentes de Redes y Comunicaciones se encuentran descritos en la arquitectura general “Arquitectura Tecnológica Redes y Comunicaciones”.

Equipo	Características
Computador Cliente	Navegador: ver Anexo A
Red Hat Single Sign On Internet	Componente que se utiliza para la autenticación del aplicativo. Red Hat Single Sign-On.
Cluster de Kubernetes	Plataforma de contenedores corporativa. Aloja todos los pods de Anexos, integraciones, API Mgmt, mesh, etc. Cluster OCP instalado y operativo (masters + workers).
Master Node	Nodos que ejecutan API Server, Scheduler, Controller Manager y etcd. También alojan istiod (Service Mesh control plane). No ejecutan pods de negocio.
Control Plane	Conjunto de nodos y procesos que gobiernan el clúster Kubernetes: exponen la API del clúster, deciden dónde se ejecuta cada pod, mantienen el estado deseado y almacenan la configuración crítica (etcd). En él también reside el control plane del Service Mesh (istiod), que distribuye configuración y certificados mTLS a los sidecars. En el Control Plane no se ejecutan cargas de negocio; solo componentes de plataforma. Su indisponibilidad afecta a todo el clúster (no se pueden crear/actualizar pods, aunque los existentes sigan corriendo).
Kube Controller Manager	Kubernetes opera con un concepto de bucle de control, que concilia continuamente el estado real del clúster con el deseado. El administrador de controladores supervisa estos controladores para garantizar que las cargas de trabajo cumplan con los requisitos especificados.
Kube API Server	El servidor API de Kubernetes es el punto de entrada para interactuar con el clúster. Los usuarios se comunican con el clúster a través de esta API, que coordina las acciones de otros componentes según sea necesario.
Kube Scheduler	La función del programador es asignar cargas de trabajo (pods) a los nodos en función de los recursos disponibles, como CPU y memoria, para optimizar el rendimiento de las aplicaciones en todo el clúster.
etcd	Kubernetes utiliza etcd, un almacén de clave-valor de alta disponibilidad, como base de datos para almacenar toda la información relacionada con el clúster. Esto garantiza la consistencia de los datos de recursos en todos los nodos.
Istio / Service Mesh – Control Plane	Componente que gestiona configuración del mesh: certificados, políticas mTLS, VirtualServices, DestinationRules.
Data Plane	Conjunto de nodos worker donde realmente se ejecutan los contenedores de negocio: microservicios de Anexos, microservicios de integración, frontend, API Management, gateways de mesh. El Data Plane procesa el tráfico de usuario y de sistema; aquí viven los sidecars Envoy del Service Mesh. El diseño separa el Data Plane en pools de workers por rol (Backend, Integraciones, Frontend, Infra, Mesh) para aislar cargas, facilitar el escalado y limitar el “blast radius” ante fallos.

Equipo	Características
Istio Ingress Gateway	Termina TLS de las Rutas, expone Frontend, API Mgmt, Keycloak hacia la red Intranet.
pods	El pod es la unidad lógica más pequeña de Kubernetes. Un pod contiene uno o más contenedores que se ejecutan en un nodo de trabajo.
Worker Node - BackEnd	Worker node encargado de contener a todos los componentes de la aplicación donde se ejecutan los microservicios de dominio Anexos, con sidecar Envoy (mesh).
Microservicios Backend Gestión Anexos (Quarkus)	Servicios REST de dominio Anexos, responsivos y reactivos (Quarkus, Hibernate Reactive, Oracle client). Imágenes Docker por servicio (sri-anexos-*-service), Deployments, Services ClusterIP, configuración de health checks.
Worker Node - FrontEnd	Worker Node encargado de gestionar los pods requeridos donde se despliega el frontend Intranet (Vue.js) empaquetado como contenedor.
Frontend Intranet Gestión Anexos (Vue.js)	Aplicación web que consume APIs de Anexos a través de API Mgmt y SSO Keycloak. Imagen Docker/Nginx con artefacto estático generado por Vue; Deployment + Service + Route.
Worker Node - API Management	Worker Node encargado de gestionar los pods deseados del API Management, aislados del backend.
Worker Mesh	Worker Node, donde residen Istio Ingress Gateway y Istio Egress Gateway, que manejan todo el tráfico que entra/sale del mesh.
INTRASRI	Oracle Database versión Anexo A , instalado en un sistema operativo Solaris. Mediante el firewall de base de datos (DBF) son auditadas las acciones ejecutadas desde la aplicación Anexos y desde cualquier cliente contra esta base de datos.
JBoss EAP	Servidor de Aplicaciones Red Hat JBoss EAP y JDK de acuerdo con lo descrito en Anexo A, instalado en un Red Hat Linux, este servidor de aplicaciones destinado para desplegar los servicios web tipo REST de las integraciones y convivencias. La infraestructura requerida corresponde a 2 servidores RHEL 10.x, cada uno con 2 instancias independientes de JBoss EAP, obteniendo un total de 4 instancias JBoss EAP distribuidas entre ambos servidores.

3. Glosario

3.1. Acrónimos

ACL	Anti-Corruption Layer
API	Application Programming Interface
API Mgmt	API Management (gateway institucional)
BFF	Backend for Frontend
BI	Business Intelligence
BI_DWH	Business Intelligence – Data Warehouse
BPM/BPMS	Business Process Management Suite
CI/CD	Integración y despliegue continuo
ConfigMap	Configuración externa en Kubernetes

DAL	Data Access Layer (implícito en integraciones)
DDD	Domain-Driven Design
DNS	Domain Name System (Mesh/Services)
EAP	Enterprise Application Platform (JBoss/EAP)
EDA	Event-Driven Architecture
EFK/ELK	Stack de logging (Elasticsearch, Fluentd/Fluentbit, Kibana)
etcd	Consistent and Highly-Available Key-Value Store for Kubernetes
JDK	Java Development Kit
JSON-B	Java API for JSON Binding
JVM	Java Virtual Machine (mencionado implícitamente por contexto JDK)
K8s	Kubernetes
OAuth2	Protocolo de autorización
OIDC	OpenID Connect
R1	Release 1
Rest	Representational State Transfer
RH-SSO	Red Hat Single Sign-On
S3/ODF	Objeto distribuido institucional (Object Data Fabric)
SPA	Single Page Application
SRI	Servicio de Rentas Internas
VKS	vSphere Kubernetes Service

3.2. Definiciones

REST.- Arquitectura de servicios web que utiliza verbos HTTP estándar, lo que simplifica su implementación y comprensión. Facilita la interacción entre diferentes plataformas y sistemas.

Quarkus.- Framework de BackEnd optimizado para tiempos de arranque rápidos y bajo consumo de recursos, ideal para microservicios en Java.

Vue.js.- Framework de FrontEnd que permite construir interfaces reactivas y fáciles de desarrollar, mejorando la experiencia del usuario y la productividad del desarrollador.

Microservicios.- Arquitectura de desarrollo que descompone la aplicación en servicios pequeños y autónomos, cada uno enfocado en una capacidad de negocio específica. Mejora el modularidad, escalabilidad y el despliegue independiente.

Service Mesh.- es una capa de infraestructura que maneja la comunicación entre microservicios Descubrimiento de servicios, proporcionando las siguientes funciones:

- Balanceo de carga.
- Resiliencia
- Seguridad
- Monitoreo y trazabilidad

Event-Driven Architecture (EDA).- Modelo arquitectónico que permite a los componentes del sistema reaccionar de forma asincrónica a eventos. Mejora el desacoplamiento, la escalabilidad horizontal y la resiliencia.

OpenAPI.- Especificación para la definición y documentación de APIs RESTful y SOAP. Permite estandarización e interoperabilidad.

Kubernetes.- Plataforma de orquestación de contenedores que permite desplegar, escalar y gestionar aplicaciones en contenedores de forma automatizada y resiliente.

Pod.- Unidad lógica más pequeña en Kubernetes, que contiene uno o más contenedores ejecutándose en un nodo.

Node (Nodo).- Servidor físico o virtual dentro de un clúster de Kubernetes donde se ejecutan los pods.

Control Plane.- Componente de Kubernetes que administra el clúster y toma decisiones globales, como la programación de pods y la detección de eventos.

Data Plane.- Donde se ejecutan las aplicaciones del usuario, es decir, las cargas de trabajo en los nodos de trabajo.

Kubelet.- Agente que se ejecuta en cada nodo de Kubernetes y gestiona el ciclo de vida de los contenedores.

kube-proxy.- Componente que gestiona el tráfico de red entre pods en diferentes nodos dentro del clúster.

Etcd.- Base de datos clave-valor distribuida utilizada por Kubernetes para almacenar la configuración del clúster de forma consistente.

API Server.- Componente central de Kubernetes que expone la API y sirve de punto de entrada para la gestión del clúster.

Kube Scheduler.- Encargado de asignar pods a nodos disponibles según recursos y políticas definidas.

Namespace.- Contenedor lógico para organizar y aislar recursos en almacenamiento de objetos, compatible con protocolos como Amazon S3.

ACID.- es un conjunto de propiedades que garantizan la fiabilidad en las transacciones dentro de sistemas de bases de datos. El término proviene de:

- **Atomicidad (Atomicity):** Cada transacción se ejecuta como una unidad indivisible: o se completa totalmente o no se realiza en absoluto. Si ocurre un fallo, se revierte todo lo que se haya hecho.
- **Consistencia (Consistency):** Después de una transacción, la base de datos debe pasar de un estado válido a otro estado válido, respetando todas las reglas, restricciones y relaciones definidas.
- **Aislamiento (Isolation):** Las transacciones concurrentes no deben interferir entre sí. Cada transacción debe ejecutarse como si fuera la única en el sistema, evitando efectos colaterales.
- **Durabilidad (Durability):** Una vez que una transacción se confirma (commit), sus cambios persisten incluso ante fallos del sistema, gracias a mecanismos como registros (logs) y respaldos.

4. Bibliografía

The New Stack. (n.d.). How Redis simplifies microservices design patterns. The New Stack.

<https://thenewstack.io/how-redis-simplifies-microservices-design-patterns/>

Fowler, M. (2011, 14 de julio). CQRS. Martin Fowler's Bliki. <https://martinfowler.com/bliki/CQRS.html>

Newman, C. (n.d.). Saga pattern. Microservices.io. Retrieved February 21, 2025, from

<https://microservices.io/patterns/data/saga.html>

XenonStack. (2024, December 18). Microservices Architecture and Design Patterns. Retrieved from

<https://www.xenonstack.com/insights/microservices>

Matei, Z., & Xin, R. (2022). Apache Spark: Unified Engine for Big Data Processing. Apache Software

Foundation. <https://spark.apache.org/>

Oracle Corporation. (2023). Oracle Database JSON Developer's Guide.

<https://docs.oracle.com/en/database/oracle/oracle-database/21/adjsn/>

Red Hat. (2023). 3scale API Management Documentation.

https://access.redhat.com/documentation/en-us/red_hat_3scale/

5. Anexos

5.1. Anexo A: Versión de tecnologías

Componente	Versión
BackEnd	
JDK	Red Hat OpenJDK 21.0.8. con su última actualización y/o parche liberado.
Quarkus	3.36.1
Lombok	1.18.42
MapStruct	1.6.3.redhat-00001

FrontEnd	
Vue	3.5.24
Node	22.19.0
NPM	10.8.3
Vue Cli	5.0.8
TypeScript	5.9.3
Vite	7.3.2
PrimeVue	4.3.7
Primeicons	7.0.0
BootStrap	5.3.7
Pinia	3.0.4
Axios	1.16.0
Keycloak-js	26.2.0
formulajs	4.6.0
vuelidate/core	2.0.3
vuelidate/validators	2.0.4
pawel-up/jexl	4.4.2
uuid	14.0.0
chevrotain	12.0.0
vue-codemirror6	1.5.1
vuedraggable	4.0.1
read-excel-file	9.0.9
vue-i18n	11.4.5
vue-router	4.6.4
v-idle-3	0.3.14
@primeuix/themes	2.0.3
jwt-decode	4.0.0
@fontawesome/fontawesome-svg-core	7.2.0
@fontawesome/free-brands-svg-icons	7.2.0
@fontawesome/free-regular-svg-icons	7.2.0
@fontawesome/free-solid-svg-icons	7.2.0
@fontawesome/vue-fontawesome	3.2.0
@codemirror/lang-json	6.0.2
@codemirror/language	6.12.3
@codemirror/theme-one-dark	6.1.3
@codemirror/view	6.41.1
Especificación Jakarta EE 10	
JDK	Red Hat OpenJDK 21.0.8. con su última actualización y/o parche liberado.
JBoss	EAP 8.1.0. con su última actualización y/o parche liberado.

Jakarta Persistence	3.1
Jakarta Contexts and Dependency Injection	4.0
Jakarta Servlet	6.0
Jakarta Enterprise Beans	4.0
Jakarta Bean Validation	3.0
Jackson	2.17
Jakarta Interceptors	2.1
Jakarta Transactions	2.0
Jakarta Restful Web Services	JAX-RS 3.1
Base de datos	
INTRASRI	Oracle 19.x c Enterprise Edition
STAR	Oracle 19.20c Enterprise Edition
Red Hat Single Sign On	7.1.4.GA
Linux	Red Hat 10.x con su última actualización y/o parche liberado
Navegadores	• Mozilla Firefox 146.0 o Superior • Google Chrome 110 o Superior • Microsoft Edge 143.0 o Superior

5.2. Anexo B: Matriz de integraciones y convivencias

La matriz se encuentra en el confluence en la siguiente ubicación:

[Matriz de Integraciones y Convivencias Anexos](#)

5.3. Anexo C: Estructura librería pistas-auditoria-lib v1.0.0

Figura que define la estructura e información del componente JAR de la librería de pistas de auditoría.

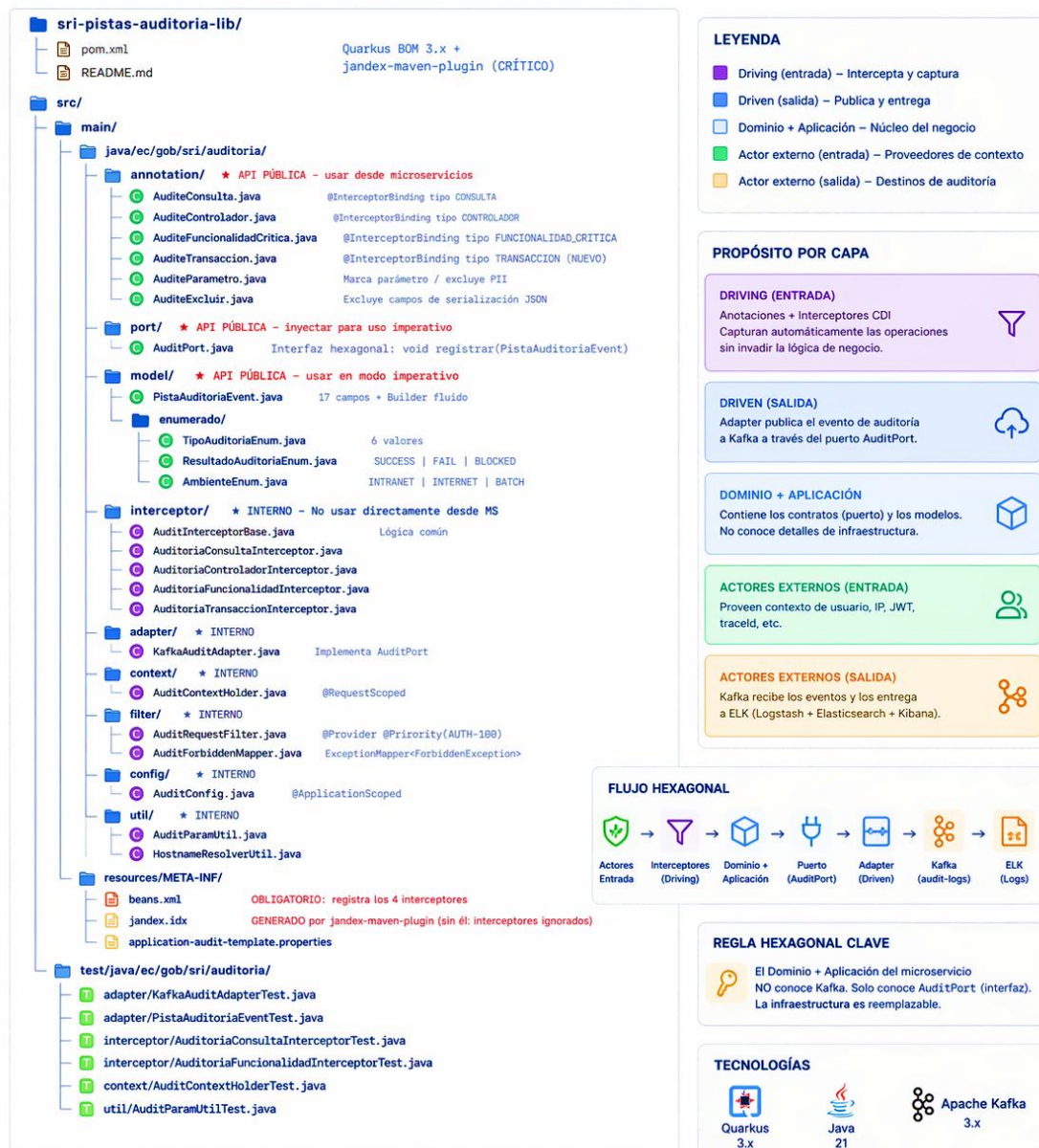


Figura 7. Estructura de librería pistas de auditoria

Responsabilidades por paquete

PAQUETE	TIPO	RESPONSABILIDAD	VISIBILIDAD DESDE MS CONSUMIDOR
annotation/	API PÚBLICA	5 anotaciones @InterceptorBinding. El desarrollador las aplica en Application Services	Usar directamente
port/	API PÚBLICA	AuditPort — contrato hexagonal. Inyectar para uso imperativo opcional	Inyectar si necesario
model/	API PÚBLICA	PistaAuditoriaEvent + 3 enumerados. Usar en modo imperativo con AuditPort	Usar en modo imperativo
interceptor/	INTERNO	4 interceptores CDI @AroundInvoke. Construyen el evento, instrumentan Uni, publican	NO usar directamente

adapter/	INTERNO	KafkaAuditAdapter implementa AuditPort. Emitter Kafka acks=all. Absorbe errores	NO usar directamente
context/	INTERNO	AuditContextHolder @RequestScoped. Aisla IP/hostname/origen por request	NO usar directamente
filter/	INTERNO	AuditRequestFilter captura headers HTTP. AuditForbiddenMapper mapea 403 a pista	NO usar directamente
config/	INTERNO	AuditConfig lee @ConfigProperty al startup. Valida campos obligatorios	NO usar directamente
util/	INTERNO	AuditParametroUtil serializa parámetros. HostnameResolverUtil resuelve hostname async	NO usar directamente

Modelo del Evento PistaAuditoria - JSON

JSON publicado al topic de Kafka.

```
{
  "usuario": "jada270709",
  "ipCliente": "10.12.1.50",
  "hostname": "pc-jada270709-01",
  "fechaHora": "2026-04-08T14:30:00.123",
  "codigoAplicacion": "GAN-PLANTILLA",
  "codigoModulo": "GAN",
  "tipoAuditoria": "FUNCIONALIDAD_CRITICA",
  "ambiente": "INTRANET",
  "codigoAplicacionOrigen": "GAN-PLANTILLA",
  "accion": "APROBAR_PLANTILLA",
  "traceld": "4bf92f3577b34da6a3ce929d0e0e4736",
  "resultado": "SUCCESS",
  "recurso": "GanPlantilla",
  "recursold": "12345",
  "mensajeError": null,
  "versionServicio": "1.0.0",
  "nombrePod": "sri-anexos-estructura-service-jvm-7d9f-xk2"
}
```